

INTRODUCTION TO ALGORITHMS AND SOFTWARE PROGRAMMING

Final Project Report - 2026

High-Throughput Limit Order Book Matching Engine

Submitted by

U Sriram Anush

SR No: 26918

Project Repository:

github.com/sriramanush105/High-Throughput-Limit-Order-Book-Matching-Engine

Contents

1	Abstract	3
2	Introduction	3
3	Background and Motivation	3
3.1	The Limit Order Book	3
3.2	Performance Requirements	4
4	System Design and Architecture	4
4.1	Core Data Structures	4
4.1.1	Price-Level Maps	4
4.1.2	Price Level as Intrusive FIFO Queue	4
4.1.3	Order Index	4
4.2	Order Book Data Structures	5
4.3	System Architecture	5
4.4	Price Representation	6
4.5	Order Matching Workflow	6
4.6	Matching Algorithm	7
4.7	Algorithmic Complexity	8
5	Implementation Details	8
5.1	Project Structure	8
5.2	Build and Execution	9
6	Dataset	9
7	Results and Performance Evaluation	10
7.1	Performance Comparison	10
7.2	Synthetic Benchmark	10
7.3	LOBSTER Replays	11
7.4	Final Book States	11

8	Discussion	11
8.1	Correctness Validation	11
8.2	Limitations	12
8.3	Comparison with Production Systems	12
9	Future Work	12
10	Conclusion	12

1. Abstract

In this project, I designed and implemented a high-throughput in-memory limit order book matching engine in C++17. The engine follows strict Price-Time Priority, which is the standard rule used by modern electronic exchanges to decide execution order.

I built the system to study how data-structure choices affect matching speed, cancellation efficiency, and overall throughput. On a synthetic benchmark, the engine achieved 5,530,165 operations per second with an average latency of 180.8 nanoseconds per operation. I also evaluated the engine on two LOBSTER NASDAQ datasets from 21 June 2012. On the MSFT replay, it processed 666,627 messages at 3,233,943 messages per second with 309.2 nanoseconds per message and emitted 32,947 trades. On the AAPL replay, it processed 400,391 messages at 2,894,502 messages per second with 345.5 nanoseconds per message and emitted 33,080 trades. To support these workloads, I used sorted price-level maps, intrusive FIFO queues, and a hash-based order index.

2. Introduction

Modern financial exchanges such as NYSE and NASDAQ handle millions of order-related events under tight latency constraints. Every incoming order, cancellation, or execution updates the limit order book, so the matching engine must respond quickly while preserving market rules.

A limit order tells the exchange to buy or sell an instrument at a specified price or better. The exchange stores active buy and sell interests in the limit order book and matches them according to price and arrival time. Because this process sits at the core of electronic trading, matching engines are a classic example of performance-critical system design.

In this project, I implemented a complete in-memory matching engine from scratch. My main goals were to enforce Price-Time Priority correctly, achieve high throughput through efficient data structures, and validate the implementation on both synthetic data and real NASDAQ replay files.

3. Background and Motivation

3.1. The Limit Order Book

A limit order book stores all active buy orders, called bids, and all active sell orders, called asks, for a given instrument. It organizes them first by price and then by time, so better prices receive priority and earlier orders at the same price execute first.

When an incoming order crosses the best available price on the opposite side, the engine executes trades immediately. This mechanism makes the order book central to liquidity, price discovery, and execution quality in modern markets.

3.2. Performance Requirements

Real exchanges process large message volumes, and many of those messages are cancellations rather than new trades. As a result, a practical matching engine must support fast best-price access, efficient cancellations, and matching behavior that scales with the number of actual fills instead of the total number of resting orders.

These requirements directly shape the implementation strategy. I therefore focused on data structures that minimize lookup time, reduce pointer chasing, and preserve FIFO order with low overhead.

4. System Design and Architecture

4.1. Core Data Structures

I built the engine around three connected data structures. Each one handles a specific performance requirement, and together they support insertion, matching, cancellation, and book maintenance efficiently.

4.1.1 Price-Level Maps

I used two sorted maps to maintain the bid side and ask side of the book. The bid map stores prices in descending order, so the best bid stays at the front, while the ask map stores prices in ascending order, so the best ask is always easy to access.

This arrangement gives constant-time access to the top of book through the front iterator and logarithmic-time insertion or removal of price levels. It also keeps the matching logic simple because the engine always starts from the best available opposite price.

4.1.2 Price Level as Intrusive FIFO Queue

At each price level, I store orders in a doubly linked list that preserves arrival order. I keep the next and previous pointers inside the order object itself, which makes the list intrusive and avoids separate node allocation.

This choice improves cache locality and reduces allocation overhead. It also gives $O(1)$ append-to-tail, $O(1)$ removal from the head, and $O(1)$ removal of an arbitrary order during cancellation.

4.1.3 Order Index

I maintain an unordered hash map from order ID to the corresponding order location in the book. This index lets the engine find active orders quickly without traversing price levels or FIFO queues.

Because of this design, cancellation and quantity reduction run in average $O(1)$ time. That matters in practice because cancellation-heavy workloads are common in real market data.

4.2. Order Book Data Structures

The following figure summarizes how the bid map, ask map, and order index interact inside the order book.

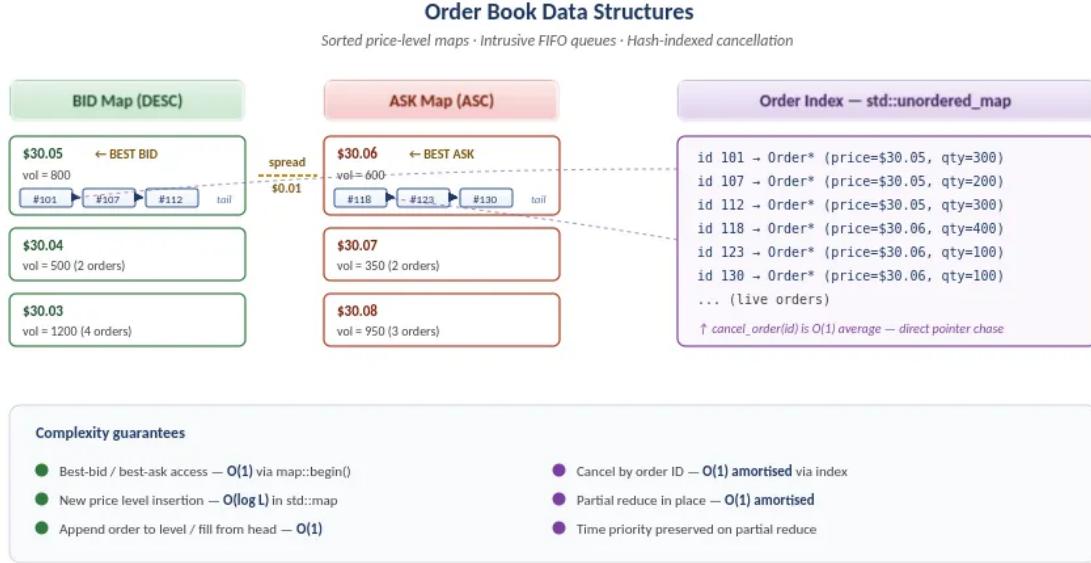


Figure 1: Order book data structures and complexity guarantees.

The figure helps connect the implementation with the complexity claims. It also makes the cancellation path and FIFO organization easier to understand than a purely textual explanation.

4.3. System Architecture

The system accepts two main workload sources: a synthetic benchmark generator and a LOBSTER replay driver. Both send events into the same matching engine core, which keeps the implementation modular and ensures that every evaluation uses the same underlying logic.

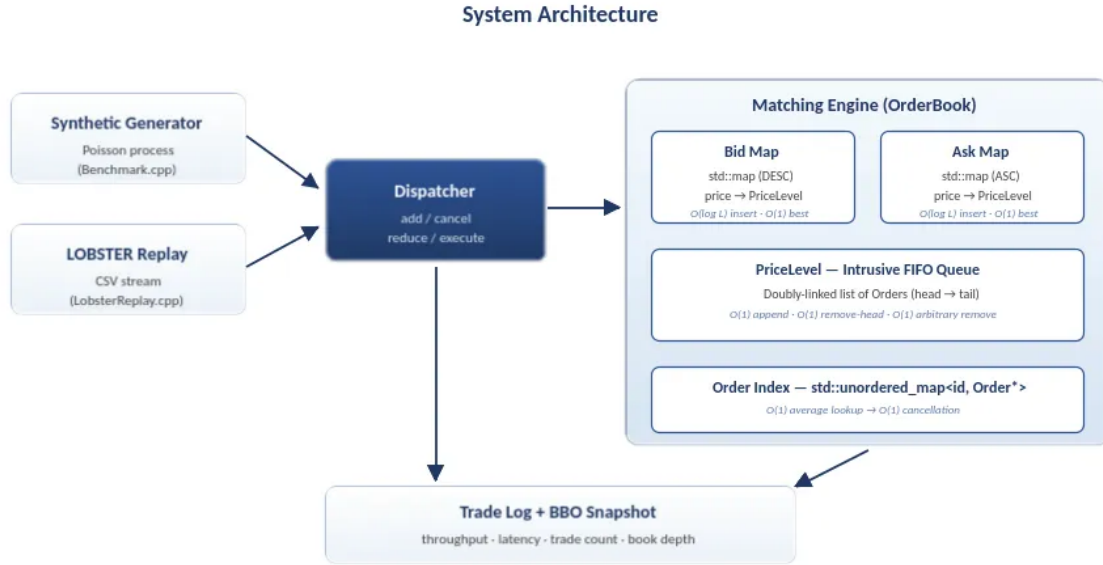


Figure 2: System architecture of the matching engine.

This architecture makes the project easier to test and reason about. It clearly separates workload generation, parsing, dispatch, matching, and output handling.

4.4. Price Representation

I store all prices as 64-bit integers using the LOBSTER convention, where a dollar price is multiplied by 10,000. This avoids floating-point comparison issues and guarantees that orders with the same displayed price map to the same internal level.

4.5. Order Matching Workflow

The matching path is the most performance-sensitive part of the engine. For every incoming order, the engine first checks whether the order crosses the best price on the opposite side. If it does, the engine enters the matching loop, executes trades against the oldest resting orders at the best price, and continues until the incoming quantity is exhausted or the book no longer crosses.

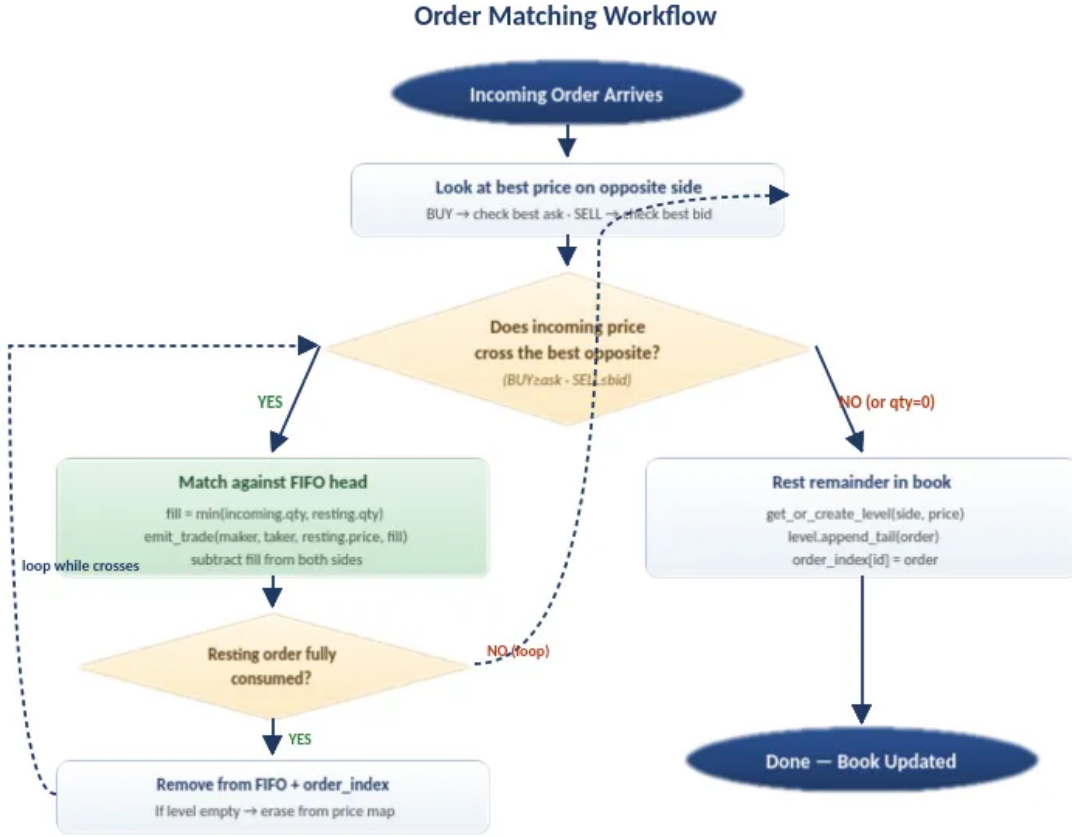


Figure 3: Order matching workflow in the engine.

If quantity remains after matching, the engine inserts the residual order into the appropriate price level. Cancellations and reductions bypass the matching loop and directly update the stored order through the order index.

4.6. Matching Algorithm

The algorithm enforces Price-Time Priority at every step. It always matches against the best available opposite price first, and within a price level it always consumes the oldest resting order before newer ones.

I record each trade at the maker's price, which is consistent with exchange-style book semantics. This rule matters because it preserves the economic meaning of passive liquidity resting in the book.

4.7. Algorithmic Complexity

Operation	Complexity	Data Structure Used
Insert new order	$O(\log L)$	Sorted price-level map
Find best bid / ask	$O(1)$	Front iterator of map
Append order to level	$O(1)$	Doubly linked list
Fill at price level	$O(1)$ per fill	Remove head of list
Cancel by order ID	$O(1)$ amortized	Hash map + list unlink
Partial reduce	$O(1)$ amortized	In-place mutation
Erase empty level	$O(\log L)$	Sorted map erase

Table 1: Time complexity of major engine operations, where L is the number of active price levels.

These complexity bounds explain why the engine remains efficient even when the book becomes large. The design keeps expensive work localized to fills and price-level updates rather than broad scans of the full book.

5. Implementation Details

5.1. Project Structure

I organized the project as a single-binary C++17 application with separate files for orders, price levels, book logic, replay, benchmarking, and the command-line entry point.

File	Responsibility
Order.h	Defines the order object with price, quantity, ID, side, timestamp, and intrusive pointers
PriceLevel.h	Implements a FIFO queue for one price level with constant-time append and removal
OrderBook.h / .cpp	Implements matching logic, bid/ask maps, order index, cancellations, and modifications
LobsterReplay.h/.cpp	Parses LOBSTER CSV messages and drives replay into the engine
Benchmark.h / .cpp	Generates synthetic workloads and measures runtime
main.cpp	Provides command-line entry points for demo, bench, and replay modes

Table 2: Source file organization.

5.2. Build and Execution

The project builds with a standard Makefile and a C++17 compiler. It supports three execution modes:

- **demo** — runs a small hand-crafted scenario to verify matching and cancellation behavior.
- **bench** [num_ops] — runs the synthetic benchmark, with one million operations as the default.
- **replay** <csv_path> — replays a LOBSTER message file through the engine.

This setup made testing straightforward during development. I could first debug behavior on the small demo, then stress the engine with the benchmark, and finally validate it on real exchange-style message streams.

6. Dataset

To validate correctness and performance, I used LOBSTER data, which reconstructs NASDAQ limit order book activity from market messages. I ran replay experiments on two files from 21 June 2012: one for Microsoft (MSFT) and one for Apple (AAPL).

The replay driver maps each LOBSTER message type to an internal engine action. This mapping lets the engine process real market activity while keeping the internal interface clean and explicit.

Message Type	LOBSTER Code	MSFT Count	AAPL Count	Meaning
New limit order submission	Type 1	328,670	191,015	Add order
Partial cancellation	Type 2	5,559	3,260	Reduce quantity
Full deletion	Type 3	299,226	171,126	Remove order
Visible execution	Type 4	29,608	23,658	Execution event
Hidden execution	Type 5	3,564	11,332	Hidden liquidity execution
Trading halt	Type 7	0	0	Halt event
Total rows processed	—	666,627	400,391	100% of file

Table 3: LOBSTER message types and counts for the replay datasets.

7. Results and Performance Evaluation

7.1. Performance Comparison

The following figure summarizes the benchmark and replay results. It provides a compact visual comparison across the synthetic workload and the two real-market datasets.

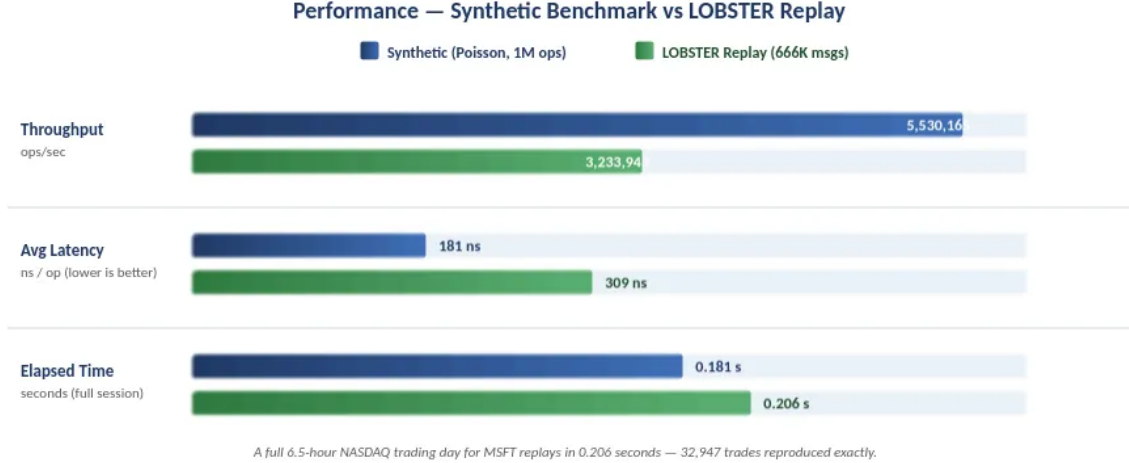


Figure 4: Performance comparison between the synthetic benchmark and LOBSTER replays.

7.2. Synthetic Benchmark

I generated one million synthetic operations using a Poisson-style arrival model with interleaved submissions and cancellations. I centered prices near a mid-price of \$31.00 and measured runtime using `high_resolution_clock`.

Metric	Synthetic Benchmark	MSFT Re-play	AAPL Re-play	Unit
Throughput	5,530,165	3,233,943	2,894,502	ops/sec or msgs/sec
Average latency	180.8	309.2	345.5	ns/op or ns/msg
Elapsed time	0.181	0.206	0.138	seconds
Trades emitted	471,286	32,947	33,080	count
Messages / rows	1,000,000	666,627	400,391	count

Table 4: Benchmark and replay performance results.

The synthetic benchmark ran faster than either replay workload. That result is expected because the synthetic stream has a more controlled message mix and avoids some of the irregular state transitions that appear in real market data.

7.3. LOBSTER Replays

I replayed the engine on both the MSFT and AAPL LOBSTER files. The MSFT run processed 666,627 rows in 0.206 seconds, which corresponds to 3,233,943 messages per second and 309.2 nanoseconds per message. The AAPL run processed 400,391 rows in 0.138 seconds, which corresponds to 2,894,502 messages per second and 345.5 nanoseconds per message.

Although the AAPL file contains fewer rows, it produced lower throughput and higher latency than MSFT. This suggests that the AAPL message stream generated a more expensive processing pattern, likely because of differences in message composition and final book structure.

Across the two real-market replays, the engine emitted 32,947 trades for MSFT and 33,080 trades for AAPL. These results show that the implementation remains stable across two distinct large-cap stocks with different market behavior.

7.4. Final Book States

After replay finished, the two books ended in noticeably different states.

Metric	MSFT			AAPL		
Best bid	\$30.13 shares	×	38,686	\$577.54	×	410 shares
Best ask	\$30.14 shares	×	10,459	\$577.67	×	400 shares
Bid-ask spread	\$0.01			\$0.13		
Active bid levels	17			14		
Active ask levels	108			844		
Live orders	4,434			2,303		
Trades emitted	32,947			33,080		

Table 5: Final book states after the MSFT and AAPL replays.

MSFT ended with a tight one-cent spread and moderate visible depth on both sides. AAPL ended with a wider spread and a much larger number of ask levels, which points to a more fragmented sell-side structure at the end of the replay.

8. Discussion

8.1. Correctness Validation

I validated the engine in two complementary ways. First, the synthetic benchmark stressed the insertion, matching, and cancellation paths at scale. Second, the MSFT

and AAPL replays showed that the engine can process full-session LOBSTER message streams while maintaining consistent book evolution.

The replay runs emitted 32,947 trades for MSFT and 33,080 trades for AAPL. Together, these experiments provide strong evidence that the implementation is functionally correct and robust enough to handle realistic market-style workloads.

8.2. Limitations

The current engine does not support stop orders, iceberg orders, or market orders. The implementation is single-threaded and handles one symbol per replay. It also still depends on the system allocator, so an object pool could reduce latency variance further. In addition, the current version does not include kernel-bypass networking or production-grade infrastructure around the matcher.

8.3. Comparison with Production Systems

Production matching engines at major exchanges operate with highly optimized software and hardware stacks. They often use kernel-bypass networking, CPU pinning, huge pages, custom memory management, and carefully engineered concurrency models.

My project does not aim to replicate a full exchange stack. Instead, it focuses on the algorithmic core of matching. Even so, the structure of the engine closely follows real-world principles, so it provides a strong baseline for further systems work.

9. Future Work

Several extensions would make the engine more complete and closer to production quality:

- Add object pooling to reduce allocation overhead and latency variance.
- Broadcast top-of-book updates after every state change.
- Support additional order types such as IOC, stop-limit, and iceberg orders.
- Extend the system to multiple symbols with shared infrastructure.
- Explore lock-free or parallel designs for higher concurrency.

10. Conclusion

In this project, I built a high-throughput in-memory limit order book matching engine in C++17 and evaluated it on both synthetic and real replay workloads. The design combines sorted price maps, intrusive FIFO queues, and a hash-based order index to support efficient insertion, cancellation, and matching.

The final results show that the engine achieves 5.53 million operations per second on the synthetic benchmark, 3.23 million messages per second on the MSFT replay, and 2.89 million messages per second on the AAPL replay. Across both real-market datasets, it maintains sub-microsecond average latency while processing large message volumes and preserving realistic final book states. These results show that careful data-structure choices can deliver strong matching performance even in a compact single-process implementation.

References

- [1] Gould, M. D., Porter, M. A., Williams, S., McDonald, M., Fenn, D. J., & Howison, S. D. (2013). *Limit order books*. Quantitative Finance, 13(11), 1709–1742.
- [2] Cont, R. (2011). *Statistical modeling of high-frequency financial data*. IEEE Signal Processing Magazine, 28(5), 16–25.
- [3] Harris, L. (2003). *Trading and Exchanges: Market Microstructure for Practitioners*. Oxford University Press.
- [4] Huang, W., Lehalle, C.-A., & Rosenbaum, M. (2015). *Simulating and Analyzing Order Book Data*. Journal of the American Statistical Association, 110(509), 107–122.
- [5] LOBSTER Academic Data. Humboldt University Berlin. <https://lobsterdata.com>